

Salesforce

Salesforce Equivalent of Implement Client Script & UI Policy (Incident) — ServiceNow

1. Overview

This project was built on ServiceNow, using UI Policies and Client Scripts on the Incident table to make fields mandatory, auto-populate values, control field behavior, and block invalid saves. Salesforce is a different CRM/platform, but it offers its own declarative and programmatic tools that achieve the same kind of data-integrity goals on a record like a Case (Salesforce's equivalent of an Incident).

This document maps each ServiceNow component used in this project to its closest Salesforce equivalent, so the same business requirement — consistent, validated Incident/Case data — could be reproduced on the Salesforce platform.

2. Component-by-Component Mapping

UI Policy → Validation Rules / Dynamic Forms & Page Layouts

The "High Impact Control" UI Policy in this project made Assignment Group mandatory and Urgency read-only when Impact was High. In Salesforce, this is done with a Validation Rule (e.g., blocking save if Priority = 'High' AND Owner is blank) combined with Dynamic Forms / Page Layout field properties to conditionally show fields as read-only based on record values.

The screenshot shows the ServiceNow UI Policy configuration page for 'High Impact Control'. The interface includes a left-hand navigation pane with categories like 'System Mobile', 'Mobile UI', 'Service Catalog', 'Catalog Administration', 'System Diagnostics', 'Session Debug', 'Debug UI Policies', 'Disable UI Policies Debug', 'System UI', 'UI Policies', 'MID Server', 'MID Security Policy', 'System Web Services', and 'SOAP Security Policies'. The main content area is titled 'UI Policy - High Impact Control' and shows the configuration details. Key elements include: a 'Table' dropdown set to 'Incident [Incident]', an 'Application' dropdown set to 'Global', an 'Active' checkbox checked, a 'Short description' field containing 'High Impact Control', an 'Order' field set to 100, a 'When to Apply' section with a 'Script' tab, a 'Conditions' section with a filter condition 'Impact is 1 - High', a 'Reverse if false' checkbox checked, and a 'Global' checkbox checked. There are also buttons for 'Add Filter Condition', 'Add OR Clause', 'AND', 'OR', and 'X'.

ServiceNow UI Policy condition (Impact is 1 - High) — in Salesforce this becomes a Validation Rule formula plus a Dynamic Forms visibility condition

UI Policy Actions → Field-Level Security & Conditional Visibility

The two UI Policy Actions in this project (Assignment Group → Mandatory, Urgency → Read-only) map to Salesforce's Dynamic Forms component visibility rules, which can mark a field as read-only or required directly on the Lightning Record Page based on other field values — without any code.

onChange Client Script → Flow (Record-Triggered, Screen Flow) or Apex

The onChange script that auto-sets Urgency to High when Impact becomes High is a classic use case for a Salesforce Flow: a record-triggered Flow (or a screen Flow launched from a Quick Action) can update a field automatically the moment another field changes, the same way `g_form.setValue()` does in ServiceNow.

The screenshot shows the ServiceNow interface for configuring a Client Script. The script is named "Auto set urgency for high impact" and is associated with the "Incident" table. The UI Type is "Desktop", the Type is "onChange", and the Field name is "Impact". The script is active and global. The script code is as follows:

```

1 function onChange(control, oldValue, newValue, isLoading) {
2   if (isLoading || newValue == '') {
3     return;
4   }
5   if (newValue == '1') { // High Impact
6     g_form.setValue('urgency', '1');
7     g_form.showFieldMsg(
8       'Impact',
9       'High Impact Incidents must have High urgency and an assignment group.',
10      'info'
11    );
12  }
13 }
14 }
15

```

The "Isolate script" checkbox is checked.

ServiceNow onChange Client Script auto-setting Urgency — in Salesforce this logic is typically built as a record-triggered Flow with an Update Records action

onSubmit Client Script → Validation Rule

The onSubmit script that blocks the save when Impact is High and Assigned To is empty is functionally identical to a Salesforce Validation Rule: `AND(Priority = 'High', ISBLANK(OwnerId))` with an error message shown on the Owner field, preventing the Case from saving until it is corrected.

The screenshot shows the ServiceNow interface for configuring a Client Script. The script is named "Prevent save if assigned to missing" and is associated with the "Incident" table. The UI Type is "Desktop", the Type is "onSubmit", and the Field name is "Impact". The script is active and global. The script code is as follows:

```

1 function onSubmit() {
2   if (g_form.getValue('Impact') == '1' &&
3       g_form.getValue('assigned_to') == '') {
4     g_form.showErrorMessage(
5       'assigned_to',
6       'Assigned To is mandatory for High Impact Incidents.'
7     );
8     return false;
9   }
10   return true;
11 }
12 }
13

```

The "Isolate script" checkbox is checked.

ServiceNow onSubmit Client Script requiring Assigned To — in Salesforce, a Validation Rule performs the equivalent save-time check

onCellEdit Client Script → Inline Edit Restrictions / Field-Level Security

The onCellEdit script that blocks direct State changes from the Incident list view has a close Salesforce equivalent in disabling inline editing for a field on list views, or using Field-Level Security / a Validation Rule that specifically rejects list-view-only inline edits, forcing the user to open the record.

3. Behavior Comparison Table

- Mandatory field on condition — ServiceNow: UI Policy Action (Mandatory: true) → Salesforce: Validation Rule + Required checkbox / Dynamic Forms
- Read-only field on condition — ServiceNow: UI Policy Action (Read only: true) → Salesforce: Dynamic Forms visibility rule or Field-Level Security
- Auto-set a field value — ServiceNow: onChange Client Script (g_form.setValue) → Salesforce: Record-Triggered Flow (Update Records)
- Block save on invalid data — ServiceNow: onSubmit Client Script (g_form.showErrorBox) → Salesforce: Validation Rule (error message)
- Restrict list/inline edits — ServiceNow: onCellEdit Client Script (callback(false)) → Salesforce: Disable inline editing / Field-Level Security

4. Testing Parity

The same five test activities carried out in this project (mandatory enforcement, successful save, reverse condition, list edit blocking, form-based update) would be verified in Salesforce the same way: create/edit a Case with Priority = High and Owner blank to confirm the Validation Rule blocks save; fill Owner and save to confirm success; change Priority back to Medium/Low to confirm the Validation Rule no longer applies; and attempt an inline list edit versus a normal record edit to compare behavior.

ServiceNow validation blocking save for a missing Assigned To — the same scenario a Salesforce Validation Rule error message would cover for a Case Owner

5. Key Differences to Note

- ServiceNow Client Scripts run purely on the client (browser) for onChange/onSubmit/onCellEdit; Salesforce Flow and Validation Rules run on the server, so there is no separate "client-side only" layer — logic is more centralized.
- ServiceNow UI Policies are reversible by design ("Reverse if false"); in Salesforce, achieving the same automatic reversal typically needs the Flow or Validation Rule to be re-evaluated on every save/change, which happens naturally since Validation Rules always re-check current values.
- ServiceNow's onCellEdit script directly intercepts list-view inline edits with custom script; Salesforce more commonly disables inline editing for a field/object rather than writing custom logic for it.

6. Conclusion

Although this project was implemented on ServiceNow using UI Policies and Client Scripts, the same business requirement — enforcing mandatory fields, auto-populating values, and blocking incomplete Incident/Case data — is fully achievable on Salesforce using Validation Rules, Dynamic Forms, and record-triggered Flows. The two platforms use different tools and terminology, but the underlying goal of reliable, validated record data before submission stays the same.